

Desafio Prático: CI e Infraestrutura

Objetivo: Este desafio tem como finalidade avaliar a capacidade de arquitetar e automatizar a infraestrutura de duas APIs Node.js. O ecossistema deverá ser containerizado com Docker e orquestrado via Docker Compose, englobando a aplicação, o banco de dados e o servidor de CI (Jenkins). Além disso, deverá ser implementada uma esteira de Integração Contínua (CI) declarativa abordando qualidade, segurança e testes.

Abaixo estão os requisitos e guias do que deve ser pesquisado e implementado em cada etapa do projeto. Nenhum código pronto será fornecido; você deverá construir as soluções com base na documentação oficial e boas práticas.

1. Orquestração e Infraestrutura (Docker Compose)

Você precisará criar um arquivo de orquestração que suba três serviços interconectados. Pesquise como estruturar um `docker-compose.yml` contemplando:

Serviço: Jenkins Server

- **Build customizado:** O Jenkins não deve usar apenas a imagem padrão. Ele precisa de um Dockerfile próprio.
- **Portas:** Pesquise quais portas padrão o Jenkins utiliza para a interface web e para agentes, e mapeie-as para o host.
- **Persistência:** Utilize *volumes* para garantir que os dados e configurações do Jenkins não sejam perdidos caso o container reinicie.
- **Docker in Docker:** Pesquise como mapear o `docker . sock` do host para dentro do container do Jenkins. Isso é crucial para que o Jenkins consiga realizar o *build* das imagens da API.

Serviço: Banco de Dados

- **Imagem:** Utilize uma imagem relacional (ex: MySQL 8.0).
- **Variáveis de Ambiente:** Configure credenciais de acesso raiz e a criação de um banco de dados inicial exclusivo para os testes.
- **Persistência:** Mapeie um volume para o diretório de dados do banco, garantindo que as informações sobrevivam ao ciclo de vida do container.

Serviço: API Node.js

- **Build:** Deve apontar para o Dockerfile da própria aplicação.
- **Dependência de inicialização:** Pesquise como fazer o container da API aguardar ou depender da inicialização do banco de dados antes de subir (cláusula `depends_on`).
- **Variáveis de Ambiente:** Injete as variáveis de conexão com o banco (Host, Usuário, Senha, Nome do Banco) para que a aplicação consiga se comunicar na rede interna do Docker.

2. Construção das Imagens (Dockerfiles)

O ecossistema exigirá a criação de duas imagens personalizadas. Estude os comandos básicos de construção (FROM, WORKDIR, COPY, RUN, EXPOSE, CMD).

Dockerfile da API Node.js

- Utilize uma imagem base leve (ex: versão *alpine* do Node).
- Defina o diretório de trabalho padrão.
- Otimize o cache do Docker copiando primeiro os arquivos de manifesto (ex: `package.json`) e instalando as dependências de produção antes de copiar o restante do código.
- Exponha a porta de comunicação correta e defina o comando de inicialização da API.

Dockerfile do Jenkins

- Parta da imagem oficial LTS do Jenkins.
- Pesquise como alternar temporariamente para o usuário `root` dentro do container para instalar pacotes adicionais do sistema operacional.
- Adicione os repositórios e instale a CLI do Docker (`docker-ce-cli`). Isso dará ao Jenkins o poder de executar comandos Docker.
- Não se esqueça de retornar ao usuário `jenkins` após a instalação por motivos de segurança.

3. Esteira de Integração Contínua (Pipeline Jenkins)

Crie um arquivo `Jenkinsfile` utilizando a sintaxe Declarativa. Pesquise como definir `agent`, `tools`, `triggers` e `stages`.

Configurações Globais

- **Triggers:** Configure a pipeline para ser ativada automaticamente mediante eventos de *push* no repositório (integração com GitHub/GitLab).
- **Tools:** Garanta que a ferramenta Node.js esteja injetada e disponível para os passos da esteira.

Estágios (Stages) da Pipeline

- **Checkout:** Estágio inicial para clonar/baixar o código fonte do repositório.
- **Build / Instalação:** Comando para instalar todas as dependências do projeto Node.js de forma automatizada (ex: gerenciadores de pacotes como npm ou yarn).
- **SAST (Segurança):** Pesquise como rodar ferramentas de *Static Application Security Testing*. Dica: O próprio ecossistema Node possui comandos nativos de auditoria de pacotes para buscar vulnerabilidades. Certifique-se de tratar os retornos para que a pipeline lide adequadamente com as falhas.
- **Lint & Quality:** Implemente a execução de uma ferramenta de análise estática de código (como ESLint). Pesquise os parâmetros de linha de comando para apontar aos arquivos fonte, definir o ambiente (Node) e a versão do ECMAScript.
- **Testes:** Estágio para execução da suíte de testes automatizados configurados na API.

Ações Pós-Pipeline (Post)

- Pesquise o bloco post declarativo do Jenkins.
- **Sempre:** Limpe a área de trabalho (Workspace) do agente para evitar lixo e falta de espaço em execuções futuras.
- **Sucesso/Falha:** Adicione rotinas de log ou notificação indicando se a execução da esteira passou em todas as verificações ou se quebrou em algum ponto.